

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ – ĐẠI HỌC
QUỐC GIA HÀ NỘI**

VIỆN TRÍ TUỆ NHÂN TẠO



**BÁO CÁO MÔN HỌC
ỨNG DỤNG AI CHO NGÔN NGỮ**

**ĐỀ TÀI: ỨNG DỤNG RAG, BM25 VÀ
FAISS TRONG TRUY XUẤT THÔNG TIN
PREMIER LEAGUE**

Nhóm sinh viên thực hiện:

23020385 – Nguyễn Gia Khánh

23020427 – Vũ Văn Tới

23020439 – Nguyễn Năng Thịnh

23020433 – Mai Phan Anh Tùng

HÀ NỘI, 2025

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ – ĐẠI HỌC
QUỐC GIA HÀ NỘI
VIỆN TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI: ỨNG DỤNG RAG, BM25 VÀ
FAISS TRONG TRUY XUẤT THÔNG
TIN PREMIER LEAGUE

BÁO CÁO MÔN HỌC
ỨNG DỤNG AI CHO NGÔN NGỮ

Nhóm sinh viên thực hiện:

23020385 – Nguyễn Gia Khánh

23020427 – Vũ Văn Tới

23020439 – Nguyễn Năng Thịnh

23020433 – Mai Phan Anh Tùng

Giảng viên hướng dẫn: Ths. Ngô Minh Hương

HÀ NỘI, 2025

Mục lục

1	Vấn đề và mục tiêu nghiên cứu	3
1.1	Đặt Vấn Đề	3
1.2	Mục Tiêu Nghiên Cứu	4
1.2.1	Mục Tiêu Tổng Quát	4
1.2.2	Mục Tiêu Cụ Thể	4
2	Xây Dựng Và Triển Khai Hệ Thống	6
2.1	Mục tiêu xây dựng hệ thống	6
2.2	Kiến trúc tổng thể của hệ thống	7
2.3	Môi trường thực thi và thư viện sử dụng	7
2.4	Tổ chức mã nguồn	8
2.5	Quy trình xây dựng dữ liệu và chỉ mục	8
2.5.1	Thu thập dữ liệu gốc	8
2.5.2	Tiền xử lý và làm sạch	9
2.5.3	Tạo chunk	9
2.5.4	Chuẩn hoá corpus và lập chỉ mục	9
2.6	Sinh bộ câu hỏi – trả lời	10
2.7	Cơ chế truy hồi và sinh câu trả lời	10
2.7.1	Truy hồi BM25 và Dense	10
2.7.2	Bộ đọc có cấu trúc (StructuredReader)	11
2.7.3	Fallback sang mô hình sinh ngôn ngữ cục bộ	11
2.8	Triển khai và vận hành hệ thống	11
2.8.1	Chạy toàn bộ pipeline	11
2.8.2	Chạy từng bước	12
2.8.3	Chạy theo từng hướng truy hồi	12
2.8.4	Chế độ offline	12
2.9	Đánh giá và đầu ra	12
2.10	Kết luận	13
3	Thực nghiệm và đánh giá	14
3.1	Mục tiêu thực nghiệm	14
3.2	Thiết lập đánh giá	14
3.3	Kết quả tổng thể	14
3.4	Kết quả theo loại câu hỏi	15
3.5	Kết quả theo độ khó	15
3.6	Phân tích kết quả	15
3.7	Nhận xét chung	16

4	Kết luận và Hướng phát triển	17
4.1	Kết luận	17
4.2	Hướng phát triển	17

Chương 1

Vấn đề và mục tiêu nghiên cứu

1.1 Đặt Vấn Đề

Trong những năm gần đây, sự phát triển mạnh mẽ của Trí tuệ nhân tạo (Artificial Intelligence – AI) và các mô hình ngôn ngữ lớn (Large Language Models – LLMs) đã tạo ra nhiều bước tiến quan trọng trong lĩnh vực xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP). Một trong những ứng dụng nổi bật là các hệ thống hỏi đáp thông minh (Question Answering Systems), cho phép người dùng tương tác với dữ liệu bằng ngôn ngữ tự nhiên thay vì phải tìm kiếm thủ công thông qua các phương pháp truyền thống.

Tuy nhiên, các mô hình ngôn ngữ lớn hiện nay vẫn tồn tại một số hạn chế đáng kể. Mặc dù có khả năng sinh ngôn ngữ tự nhiên tốt, các mô hình này thường gặp hiện tượng “hallucination”, tức là tạo ra thông tin không chính xác hoặc không có trong dữ liệu thực tế. Ngoài ra, dữ liệu huấn luyện của mô hình thường bị giới hạn theo thời gian và không thể cập nhật liên tục theo dữ liệu mới. Điều này gây khó khăn khi áp dụng vào các bài toán cần tính chính xác cao và phụ thuộc vào dữ liệu chuyên ngành.

Trong lĩnh vực thể thao, đặc biệt là giải bóng đá Premier League, lượng dữ liệu về cầu thủ, đội bóng, lịch thi đấu, bảng xếp hạng, thống kê trận đấu và lịch sử mùa giải là rất lớn và liên tục thay đổi. Người dùng thường có nhu cầu tra cứu nhanh các thông tin như:

- Đội bóng nào ghi nhiều bàn thắng nhất mùa giải?
- Cầu thủ nào kiến tạo nhiều nhất?
- Thành tích đối đầu giữa hai đội?
- Thống kê số trận thắng của một câu lạc bộ?

Các hệ thống tìm kiếm truyền thống dựa trên từ khóa thường chưa đáp ứng tốt nhu cầu này do:

- Không hiểu được ngữ nghĩa của câu hỏi.
- Kết quả trả về thường là danh sách tài liệu thay vì câu trả lời trực tiếp.
- Khó xử lý các câu hỏi dài hoặc có ngữ cảnh phức tạp.

Để giải quyết vấn đề trên, mô hình Retrieval-Augmented Generation (RAG) đã được đề xuất nhằm kết hợp khả năng truy xuất thông tin với khả năng sinh ngôn ngữ của mô hình LLM. Trong kiến trúc RAG, hệ thống sẽ:

- Truy xuất các tài liệu liên quan từ cơ sở dữ liệu.
- Đưa dữ liệu truy xuất được vào mô hình ngôn ngữ.
- Sinh ra câu trả lời chính xác và có ngữ cảnh hơn.

1.2 Mục Tiêu Nghiên Cứu

1.2.1 Mục Tiêu Tổng Quát

Xây dựng hệ thống hỏi đáp thông minh về dữ liệu Premier League sử dụng mô hình Retrieval-Augmented Generation (RAG) kết hợp BM25 và FAISS nhằm nâng cao hiệu quả truy xuất thông tin và chất lượng câu trả lời.

1.2.2 Mục Tiêu Cụ Thể

Đề tài hướng đến các mục tiêu cụ thể sau:

1. Nghiên cứu lý thuyết

- Tìm hiểu về xử lý ngôn ngữ tự nhiên (NLP).
- Nghiên cứu mô hình Retrieval-Augmented Generation (RAG).
- Tìm hiểu cơ chế hoạt động của BM25.
- Tìm hiểu phương pháp vector search bằng FAISS.
- Nghiên cứu embedding và semantic search.

2. Xây dựng dữ liệu

- Thu thập dữ liệu Premier League từ các nguồn phù hợp.
- Tiền xử lý dữ liệu.
- Chuyển đổi dữ liệu thành dạng phù hợp cho truy xuất.

3. Xây dựng hệ thống truy xuất

- Xây dựng bộ truy xuất BM25.
- Xây dựng vector database bằng FAISS.
- Kết hợp Hybrid Retrieval giữa BM25 và FAISS.

4. Xây dựng phần hỏi đáp

- Tích hợp retriever với mô hình ngôn ngữ lớn.
- Sinh câu trả lời dựa trên dữ liệu truy xuất.

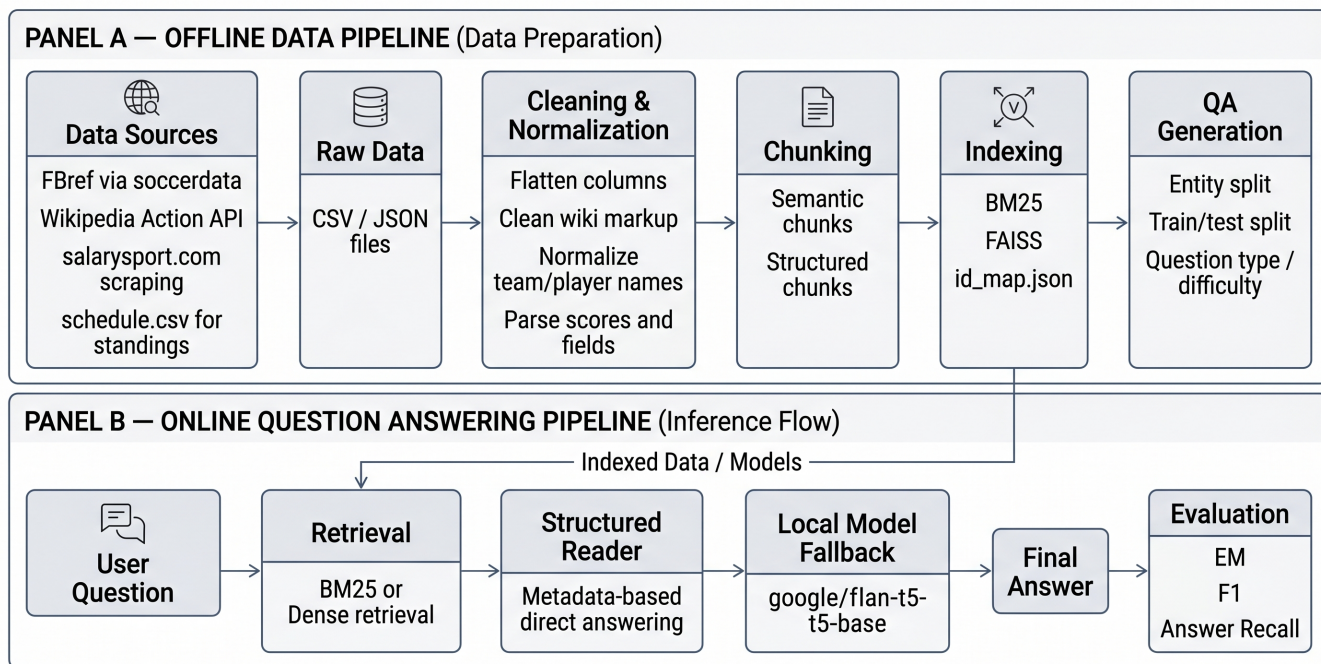
- Thiết kế pipeline RAG.

5. Đánh giá hệ thống

- Đánh giá độ chính xác truy xuất dựa trên các chỉ số.
- So sánh hiệu quả giữa: BM25, FAISS, Hybrid Retrieval.
- Đánh giá chất lượng câu trả lời của hệ thống.

Chương 2

Xây Dựng Và Triển Khai Hệ Thống



Offline data preparation and online RAG inference pipeline for the EPL 2023–24 system.

Hình 2.1: Sơ đồ mô tả quá trình hoạt động của hệ thống

2.1 Mục tiêu xây dựng hệ thống

Mục tiêu của hệ thống là xây dựng một quy trình Retrieval-Augmented Generation (RAG) hoàn chỉnh để trả lời về Giải Ngoại hạng Anh mùa giải 2023–24. Hệ thống được thiết kế theo hai pha chính:

- **Pha ngoại tuyến (offline):** thu thập dữ liệu, tiền xử lý, tạo chunk, xây dựng chỉ mục, sinh bộ câu hỏi – trả lời và chuẩn bị dữ liệu đánh giá.
- **Pha trực tuyến (online):** truy hồi dữ liệu liên quan theo câu hỏi đầu vào, dùng bộ đọc có cấu trúc để trả lời trực tiếp nếu có thể, và fallback sang mô hình sinh ngôn ngữ cục bộ khi cần thiết.

Toàn bộ hệ thống được triển khai cục bộ bằng các script Python trong thư mục `src/`, không phụ thuộc vào một dịch vụ triển khai đám mây riêng biệt.

2.2 Kiến trúc tổng thể của hệ thống

Hệ thống được tổ chức thành một pipeline tuần tự, bắt đầu từ dữ liệu gốc và kết thúc bằng câu trả lời cùng các chỉ số đánh giá. Về mặt logic, pipeline có thể chia thành các khối chức năng sau:

- Thu thập dữ liệu từ FBref, Wikipedia và salarysport.
- Chuẩn hoá và lưu dữ liệu thô trong `data/raw/`.
- Tạo các chunk semantic và structured trong `data/chunks/`.
- Chuẩn hoá corpus và xây dựng chỉ mục BM25, FAISS cùng bảng ánh xạ `id_map.json`.
- Sinh tập QA train/test theo entity split.
- Chạy pipeline RAG với hai hướng retrieval: BM25 và Dense.
- Đánh giá kết quả bằng EM, F1, Answer Recall và IAA nếu có dữ liệu annotator.

Trong đó, `pipeline.py` là thành phần suy luận chính, còn `evaluate.py` chịu trách nhiệm đo lường chất lượng đầu ra. Script `run_all.py` đóng vai trò điều phối toàn bộ chuỗi xử lý.

2.3 Môi trường thực thi và thư viện sử dụng

Hệ thống được xây dựng bằng Python và sử dụng các thư viện đúng theo mã nguồn trong repo. Các thư viện chính gồm:

- `pandas, numpy`: xử lý dữ liệu bảng và tính toán.
- `torch`: chạy mô hình sinh ngôn ngữ cục bộ.
- `transformers`: tải tokenizer và mô hình `google/flan-t5-base`.
- `sentence-transformers`: tạo embedding cho truy hồi dense.
- `rank_bm25`: xây dựng BM25 sparse retrieval.
- `faiss-cpu`: lập chỉ mục và truy hồi vector.
- `scikit-learn`: hỗ trợ các thao tác đánh giá và xử lý liên quan.
- `soccerdata`: lấy dữ liệu FBref cho cầu thủ, đội bóng và lịch thi đấu.
- `requests, beautifulsoup4`: thu thập dữ liệu Wikipedia và salarysport.

Khi chạy toàn bộ pipeline ở chế độ không có kết nối Internet hoặc muốn sử dụng cache sẵn, `run_all.py` có hỗ trợ đặt các biến môi trường:

- `HF_HUB_OFFLINE=1`
- `TRANSFORMERS_OFFLINE=1`

để đảm bảo các mô hình Hugging Face được lấy từ bộ nhớ đệm nội bộ.

2.4 Tổ chức mã nguồn

Mã nguồn của hệ thống được phân tách rõ ràng theo từng bước xử lý:

- `crawl/crawl_club_metadata.py`: lấy metadata CLB từ Wikipedia.
- `crawl/crawl_epl_transfers.py`: thu thập dữ liệu chuyển nhượng từ `salarysport`.
- `crawl/fetch_stats.py`: tải thống kê cầu thủ, đội bóng và lịch thi đấu từ FBref.
- `crawl/fetch_standings.py`: tính bảng xếp hạng từ lịch thi đấu.
- `src/chunker.py`: tạo chunk từ dữ liệu raw.
- `src/embedding.py`: chuẩn hoá corpus, tạo embedding và chỉ mục.
- `src/generate_qa.py`: sinh tập câu hỏi – trả lời và tách train/test theo entity.
- `src/pipeline.py`: truy hồi và sinh câu trả lời.
- `src/evaluate.py`: đánh giá hệ thống.
- `src/run_all.py`: điều phối toàn bộ quy trình.

Cách phân tách này giúp hệ thống dễ bảo trì, dễ kiểm thử và dễ chạy từng bước độc lập.

2.5 Quy trình xây dựng dữ liệu và chỉ mục

2.5.1 Thu thập dữ liệu gốc

Các script crawl tải dữ liệu về `data/raw/`. Cụ thể:

- **FBref qua soccerdata**: thu thập thống kê cầu thủ, đội bóng và lịch thi đấu.
- **Wikipedia Action API**: lấy metadata cơ bản của 20 câu lạc bộ EPL.
- **Web scraping từ salarysport.com**: lấy dữ liệu chuyển nhượng của từng câu lạc bộ.
- **Tính toán từ schedule**: suy ra bảng xếp hạng từ file lịch thi đấu.

Dữ liệu sau khi tải về được lưu dưới dạng CSV hoặc JSON để phục vụ các bước xử lý tiếp theo.

2.5.2 Tiền xử lý và làm sạch

Script `chunker.py` và các script crawl thực hiện các thao tác làm sạch sau:

- Làm phẳng cột đa tầng từ FBref bằng hàm `flatten_columns()`.
- Chuẩn hoá các giá trị rỗng, giá trị thiếu và kiểu dữ liệu.
- Loại bỏ wiki markup, template và thẻ tham chiếu trong dữ liệu Wikipedia.
- Chuẩn hoá tên đội, tên cầu thủ và mã nhận dạng để dễ gán nhãn và truy hồi.
- Parse tỉ số trận đấu và các trường số để phục vụ thống kê và sinh QA.

2.5.3 Tạo chunk

Sau khi dữ liệu được làm sạch, hệ thống tách dữ liệu thành các chunk nhỏ hơn để phục vụ truy hồi. Mã nguồn tạo ra hai dạng chunk:

- **Semantic chunk:** nội dung được diễn đạt tự nhiên, phù hợp cho truy hồi ngữ nghĩa.
- **Structured chunk:** nội dung theo dạng cấu trúc hoặc markdown, giúp truy hồi chính xác hơn.

Các file chunk đầu ra gồm:

- `all_players.jsonl`
- `all_teams.jsonl`
- `all_matches.jsonl`
- `transfers_chunks.jsonl`
- `schedule_chunks.jsonl`

Mỗi chunk đều đi kèm metadata để phục vụ truy xuất và tạo câu hỏi sau này.

2.5.4 Chuẩn hoá corpus và lập chỉ mục

Script `embedding.py` hợp nhất các file chunk thành một corpus thống nhất `corpus_unified.jsonl`. Từ corpus này, hệ thống xây dựng ba thành phần chính:

- **BM25 corpus:** lưu tại `data/index/bm25_corpus.pkl`.
- **FAISS index:** lưu tại `data/index/faiss_index.bin`.
- **id map:** lưu tại `data/index/id_map.json`.

Trong bước dense retrieval, hệ thống sử dụng mô hình `sentence-transformers/all-MiniLM-L6-v2` để tạo embedding cho các chunk. Vector được chuẩn hoá và lưu vào FAISS để truy hồi theo độ tương đồng cosine/inner product. Song song, BM25 được xây dựng bằng `rank_bm25` để phục vụ truy hồi sparse theo từ khoá.

2.6 Sinh bộ câu hỏi – trả lời

Script `generate_qa.py` tự động sinh bộ câu hỏi – trả lời dựa trên chunk đã tạo. Các nhóm câu hỏi được sinh gồm:

- câu hỏi về cầu thủ,
- câu hỏi về đội bóng,
- câu hỏi về trận đấu,
- câu hỏi về chuyển nhượng,
- câu hỏi tổng hợp và so sánh.

Mỗi câu hỏi được gán nhãn `type` và `difficulty`. Hệ thống sử dụng chiến lược **entity split** để tránh leakage giữa tập train và test, với tỷ lệ test mặc định là 30%.

Các đầu ra được sinh ra trong thư mục `data/qa/` gồm:

- `train/questions.txt`
- `train/reference_answers.txt`
- `train/qa_pairs.json`
- `test/questions.txt`
- `test/reference_answers.txt`
- `test/qa_pairs.json`
- `split_metadata.json`
- `iaa_subset.json`

2.7 Cơ chế truy hồi và sinh câu trả lời

2.7.1 Truy hồi BM25 và Dense

Trong `pipeline.py`, hệ thống hỗ trợ hai cách truy hồi:

- **BM25Retriever**: truy hồi sparse dựa trên token overlap.
- **DenseRetriever**: truy hồi dense bằng embedding SentenceTransformer và FAISS.

Cả hai bộ truy hồi đều trả về top- k chunk liên quan nhất cho câu hỏi đầu vào. Trong mã nguồn, giá trị `TOP_K` được đặt là 5.

2.7.2 Bộ đọc có cấu trúc (StructuredReader)

Sau khi truy hồi, hệ thống không đi thẳng vào mô hình sinh ngôn ngữ mà trước hết dùng `StructuredReader`. Thành phần này khai thác metadata trong các chunk để trả lời trực tiếp những câu hỏi có thể suy luận bằng luật hoặc tra cứu trường có cấu trúc.

Bộ đọc này có thể:

- trả lời câu hỏi về cầu thủ, đội bóng, trận đấu và chuyển nhượng,
- so sánh hai đội hoặc hai cầu thủ theo các chỉ số,
- suy luận kết quả trận đấu từ tỉ số,
- nhận diện vị trí cầu thủ theo các alias đã chuẩn hoá.

Đây là bước tối ưu quan trọng vì giúp tăng độ chính xác cho những câu hỏi đơn giản, đồng thời giảm phụ thuộc vào mô hình sinh ngôn ngữ.

2.7.3 Fallback sang mô hình sinh ngôn ngữ cục bộ

Nếu `StructuredReader` không trả lời được, hệ thống ghép các chunk đã truy hồi thành context ngắn và chuyển vào mô hình cục bộ `google/flan-t5-base`. Mô hình này được tải bằng `transformers`, và câu trả lời được sinh theo kiểu `seq2seq` nếu mô hình hỗ trợ.

Một số giới hạn được thiết lập trong mã nguồn:

- `MAX_CTX_CHARS = 800`
- `MAX_CTX_TOKENS = 400`

Nếu không có context phù hợp, hệ thống trả về thông báo không tìm thấy thông tin liên quan.

2.8 Triển khai và vận hành hệ thống

2.8.1 Chạy toàn bộ pipeline

Script `src/run_all.py` là điểm vào chính để chạy toàn bộ hệ thống theo thứ tự:

```
chunker.py → embedding.py → generate_qa.py → pipeline.py → evaluate.py
```

Người dùng có thể chạy toàn bộ pipeline bằng lệnh:

- `python src/run_all.py`

2.8.2 Chạy từng bước

Ngoài cách chạy toàn bộ, hệ thống cho phép chạy từng phần độc lập:

- `python chunker.py`
- `python embedding.py`
- `python generate_qa.py`
- `python pipeline.py`
- `python evaluate.py`

Cách này hữu ích khi cần debug từng bước hoặc chỉ muốn tái sinh một phần dữ liệu.

2.8.3 Chạy theo từng hướng truy hồi

`pipeline.py` hỗ trợ ba chế độ:

- `-approach bm25`
- `-approach dense`
- `-approach both`

Điều này cho phép so sánh trực tiếp hiệu quả giữa truy hồi sparse và dense trên cùng một bộ câu hỏi.

2.8.4 Chế độ offline

Nếu muốn chạy trong môi trường không có Internet, `run_all.py` hỗ trợ cờ `-offline`. Khi bật cờ này, hệ thống đặt:

- `HF_HUB_OFFLINE=1`
- `TRANSFORMERS_OFFLINE=1`

để mô hình Hugging Face được lấy từ cache cục bộ thay vì tải lại từ mạng.

2.9 Đánh giá và đầu ra

Script `evaluate.py` so sánh prediction với reference answers và tính các chỉ số:

- **Exact Match (EM)**
- **Token-level F1**
- **Answer Recall**

Ngoài ra, hệ thống còn thống kê theo `question type` và `difficulty`. Nếu chạy với cờ `-iaa`, script sẽ tính thêm **Cohen's Kappa** dựa trên hai file đáp án của annotator:

- `data/qa/iaa_annotator1.txt`
- `data/qa/iaa_annotator2.txt`

Các file đầu ra chính trong `outputs/` gồm:

- `system_output_bm25.txt`
- `system_output_dense.txt`
- `run_bm25_detailed.json`
- `run_dense_detailed.json`
- `evaluation_report.json`
- `evaluation_summary.txt`

2.10 Kết luận

Hệ thống RAG trong dự án được xây dựng theo quy trình rõ ràng, tách biệt giữa thu thập dữ liệu, xử lý, lập chỉ mục, sinh QA, suy luận và đánh giá. Việc tổ chức mã nguồn thành các script độc lập giúp hệ thống dễ kiểm soát, dễ tái lập và thuận lợi cho việc so sánh các chiến lược truy hồi khác nhau.

Chương 3

Thực nghiệm và đánh giá

3.1 Mục tiêu thực nghiệm

Mục tiêu của thực nghiệm là đánh giá chất lượng hệ thống RAG cho bài toán hỏi đáp về Giải Ngoại hạng Anh mùa giải 2023–24. Hai phương pháp truy hồi được so sánh gồm:

- **BM25**: truy hồi sparse dựa trên từ khóa.
- **Dense**: truy hồi dense sử dụng SentenceTransformer và FAISS.

Sau bước truy hồi, hệ thống sử dụng `StructuredReader` để trả lời trực tiếp từ meta-data. Nếu không thể suy luận bằng luật, hệ thống fallback sang mô hình sinh ngôn ngữ `google/flan-t5-base`.

3.2 Thiết lập đánh giá

Tập kiểm thử gồm **3358 câu hỏi**. Mỗi câu hỏi được chạy trên hai cấu hình:

- BM25 + StructuredReader + Flan-T5-base
- Dense + StructuredReader + Flan-T5-base

Hệ thống được đánh giá bằng ba chỉ số:

- **Exact Match (EM)**
- **Token-level F1**
- **Answer Recall**

Ngoài kết quả tổng thể, hệ thống còn được đánh giá theo:

- loại câu hỏi,
- độ khó (*easy*, *medium*, *hard*).

3.3 Kết quả tổng thể

Kết quả cho thấy cả BM25 và Dense đều đạt hiệu năng rất cao và gần như tương đương nhau. BM25 nhỉnh hơn nhẹ ở cả EM, F1 và Answer Recall, tuy nhiên mức chênh lệch rất nhỏ.

Bảng 3.1: Kết quả đánh giá tổng thể

Metric	BM25	Dense
EM	0.9866	0.9863
F1	0.9851	0.9848
Answer Recall	0.9815	0.9812

3.4 Kết quả theo loại câu hỏi

Bảng 3.2: F1 theo nhóm câu hỏi

Nhóm câu hỏi	BM25	Dense	Nhận xét
Trận đấu	1.0000	1.0000	Chính xác tuyệt đối
Đội bóng	1.0000	1.0000	Metadata ổn định
Cầu thủ	0.97–1.00	0.97–1.00	Rất cao, ổn định
So sánh	0.95–1.00	0.95–1.00	Khả năng suy luận tốt
Chuyển nhượng	0.92–1.00	0.94–1.00	Khó hơn do nhiều dạng biểu diễn

Nhóm câu hỏi về trận đấu và đội bóng đạt kết quả gần như tuyệt đối nhờ dữ liệu được tổ chức theo metadata rõ ràng. Các câu hỏi cầu thủ và so sánh cũng đạt F1 rất cao. Riêng nhóm `transfer_fee` có điểm thấp hơn do đáp án phí chuyển nhượng có nhiều cách biểu diễn khác nhau.

Nhìn chung, Dense nhỉnh hơn nhẹ ở một vài nhóm như `transfer_fee`, trong khi BM25 tốt hơn ở một số nhóm như `player_age`. Tuy nhiên sự khác biệt không đáng kể.

3.5 Kết quả theo độ khó

Bảng 3.3: F1 theo độ khó

Độ khó	BM25	Dense
easy	0.9832	0.9828
medium	0.9914	0.9871
hard	0.9883	0.9894

Kết quả cho thấy hệ thống hoạt động ổn định ở cả ba mức độ khó. Nhóm *medium* đạt điểm cao nhất do phần lớn câu hỏi vẫn có thể truy xuất trực tiếp từ metadata hoặc chunk cấu trúc.

3.6 Phân tích kết quả

Hiệu quả cao của hệ thống đến từ các yếu tố:

- Dữ liệu được tổ chức rõ ràng theo cầu thủ, đội bóng, trận đấu và chuyển nhượng.
- Chunk semantic và structured giúp truy hồi hiệu quả.

- `StructuredReader` xử lý tốt các câu hỏi truy xuất trực tiếp.
- Chuẩn hoá đáp án trước khi tính EM/F1 giúp đánh giá chính xác hơn.

3.7 Nhận xét chung

Kết quả thực nghiệm cho thấy hệ thống RAG hoạt động rất hiệu quả trên tập kiểm thử 3358 câu hỏi. BM25 và Dense đều đạt hiệu năng cao và gần như tương đương nhau. Hệ thống đặc biệt mạnh ở các câu hỏi truy xuất thông tin trực tiếp như trận đấu, đội bóng và cầu thủ, đồng thời vẫn duy trì kết quả tốt ở các câu hỏi khó hơn như so sánh và chuyển nhượng.

Chương 4

Kết luận và Hướng phát triển

4.1 Kết luận

Trong đề tài này, chúng tôi đã xây dựng thành công một hệ thống Retrieval-Augmented Generation (RAG) cho bài toán hỏi đáp về Giải Ngoại hạng Anh mùa giải 2023–24. Hệ thống tích hợp được các thành phần chính gồm thu thập dữ liệu, tiền xử lý, chunking, lập chỉ mục BM25 và FAISS, sinh bộ câu hỏi – trả lời, truy hồi thông tin và sinh câu trả lời tự động.

Kết quả thực nghiệm cho thấy hệ thống đạt hiệu quả cao trên tập kiểm thử 3358 câu hỏi, với EM, F1 và Answer Recall đều ở mức rất tốt. Điều này cho thấy cách tổ chức dữ liệu, chiến lược truy hồi và bộ đọc có cấu trúc trong hệ thống là phù hợp và ổn định.

4.2 Hướng phát triển

Trong thời gian tới, hệ thống có thể được mở rộng theo các hướng sau:

- Bổ sung thêm dữ liệu cho nhiều mùa giải khác để tăng phạm vi hỏi đáp.
- Tích hợp thêm mô hình reranking để cải thiện chất lượng truy hồi.
- Nâng cấp mô hình sinh câu trả lời bằng mô hình mạnh hơn hoặc fine-tune theo miền dữ liệu bóng đá.
- Mở rộng bộ câu hỏi để đa dạng hơn về mức độ khó và kiểu suy luận.
- Xây dựng giao diện web hoặc API để người dùng có thể đặt câu hỏi trực tiếp.

Nhìn chung, hệ thống hiện tại là một nền tảng tốt để tiếp tục phát triển thành một trợ lý hỏi đáp bóng đá hoàn chỉnh và thực tiễn hơn.